

Individual Reflection

Domi

StudyOS Thesis-Finder • individual submission • University of Tübingen

1. Initial expectation

Going into the course, I expected it to be more theory-heavy and less project-driven, and I wasn't entirely sure what to expect content-wise. What I did want was to deepen my skills in working with coding agents — both learning to work efficiently on my own with them and learning how to collaborate cleanly with a team while using these tools. I was positively surprised by the module's approach: in the first sessions we were forced to critically question what was even worth “solving,” and how to approach the task at all, before writing a line of code. The learning about the tools happened as a byproduct of actually solving the assignment, not as a separate track. The one framework that stuck with me is the double diamond: you have to question what is worth solving before spending any thought on how to solve it or what to deliver — because in the age of coding agents, the “how” has almost become secondary, since so much more is possible now that the real bottleneck moved upstream, to problem selection. Initially I assumed we'd build a fully working “assistant” for students that helps with everything, especially learning. That's technically feasible, but it misses where the actual demand is.

2. Main learning

My single most important learning is that, especially in the age of AI, the real work is figuring out which problems are worth solving — and that this is exactly the kind of question that matters in product discovery and in startups: where is there demand, where could demand emerge, and how do you even assess that? Do other people actually share my framing of the problem, or am I projecting my own view onto it? That's tied to actively seeking out other people's opinions and talking to people who have relevant experience or a valuable perspective — a single viewpoint on a problem is almost never complete. This really clicked for me when we pivoted from building an “app” to building a “skill.” No user today wants to open a separate external tool just to find a thesis topic. A lightweight skill that plugs the task directly into an LLM the user already has open solves several of these problems at once. On top of that, the simple exercise of writing “we aim to solve X for Y by doing Z” and testing it against impact, feasibility, and success already resolves a surprising number of open questions and wrong turns before they cost any implementation time.

3. Role and contribution in the group

We were a team of three with fairly similar development roles overall, but each of us leaned into a distinct strength. Max brought more expertise in building out the app itself and in evaluating the skill's performance; Valentin focused on outward communication and on user testing through his own network. My role was largely a meta-role within that setup: repeatedly stepping back and asking whether what we were currently doing actually made sense, and regularly calling for a pause to make the current state explicit and to clarify the concrete next steps for the group.

The clearest example of this dynamic was the no-DB pivot. The group was initially divided on whether a database-backed or a database-free approach made more sense — I was the one pushing for no-DB. Rather than resolve it by discussion alone, we pursued both in parallel for a short time. Once user testing came back, it quickly became clear that the no-DB approach was not only preferable from a user standpoint but also significantly less effort, especially in terms of long-term maintenance. My most concrete individual contribution was then following through on that direction: building the first no-DB prototype myself, which ultimately became the final artifact we shipped.

4. Future learning

Technically, I want to get better at working efficiently in larger AI-assisted teams — keeping communication clean, actually producing clean code, and not letting errors creep in by holding the right balance between tests and production code. I also want to keep sharpening my own “vibe coding” skill: finding the right level of communication between me and the agent so that code production runs as efficiently as possible, while still retaining enough detail and understanding to stay in control of the product rather than losing oversight of what’s being built. Beyond that, working on this project noticeably grew my interest in building my own products — seeing what coding agents are actually capable of, and getting a first real taste of how to design something with genuine demand behind it rather than just something interesting to build, made me want to pursue more of my own projects. To get there, I want to keep building expertise independently so I can take on further personal projects in the near future.